



Anton Martyniuk

Join 17,000+ Readers



N-LAYERED VS CLEAN VS VERTICAL SLICE ARCHITECTURE: CHOOSING THE RIGHT APPROACH FOR DOTNET PROJECTS IN 2025

Improve your skills on antondevtips.com

[Privacy](#) - [Terms](#)

**Anton Martyniuk**

Sep 16, 2025

 10 min read**[Sponsor my newsletter to reach 17,000+ readers](#)**

Choosing the proper project structure is one of the most impactful decisions for a .NET development team. It shapes everything from onboarding speed and code maintainability to the speed of feature delivery.

In modern software development, there are 3 popular approaches to structuring your projects:

- N-Layered Architecture (Controller-Service-Repository)
- Clean Architecture
- Vertical Slice Architecture

Over the past years, I've built and scaled .NET solutions using all three approaches. In this post, I'll break down the real trade-offs, share the pitfalls I've seen in practice, and show my personal preference that helps teams build for both speed and maintainability.

In this post, we will explore each approach with pros and cons:



Anton Martyniuk

- Vertical Slice Architecture
- Combining Clean Architecture with Vertical Slices
- The Best Architecture to use in 2025

Let's dive in.

N-Layered Architecture

The N-Layered approach is still the default in most codebases — it is widely adopted from simple to enterprise .NET applications.

It's built around separating responsibilities into logical layers, most often:

- **Presentation Layer:** Controllers, API endpoints, or UI components
- **Business Logic Layer (Service):** Application services encapsulating business rules
- **Data Access Layer (Repository):** Abstraction over data persistence

This structure is simple to understand, widely taught, and familiar to almost every .NET developer. It's common to see projects organized as:

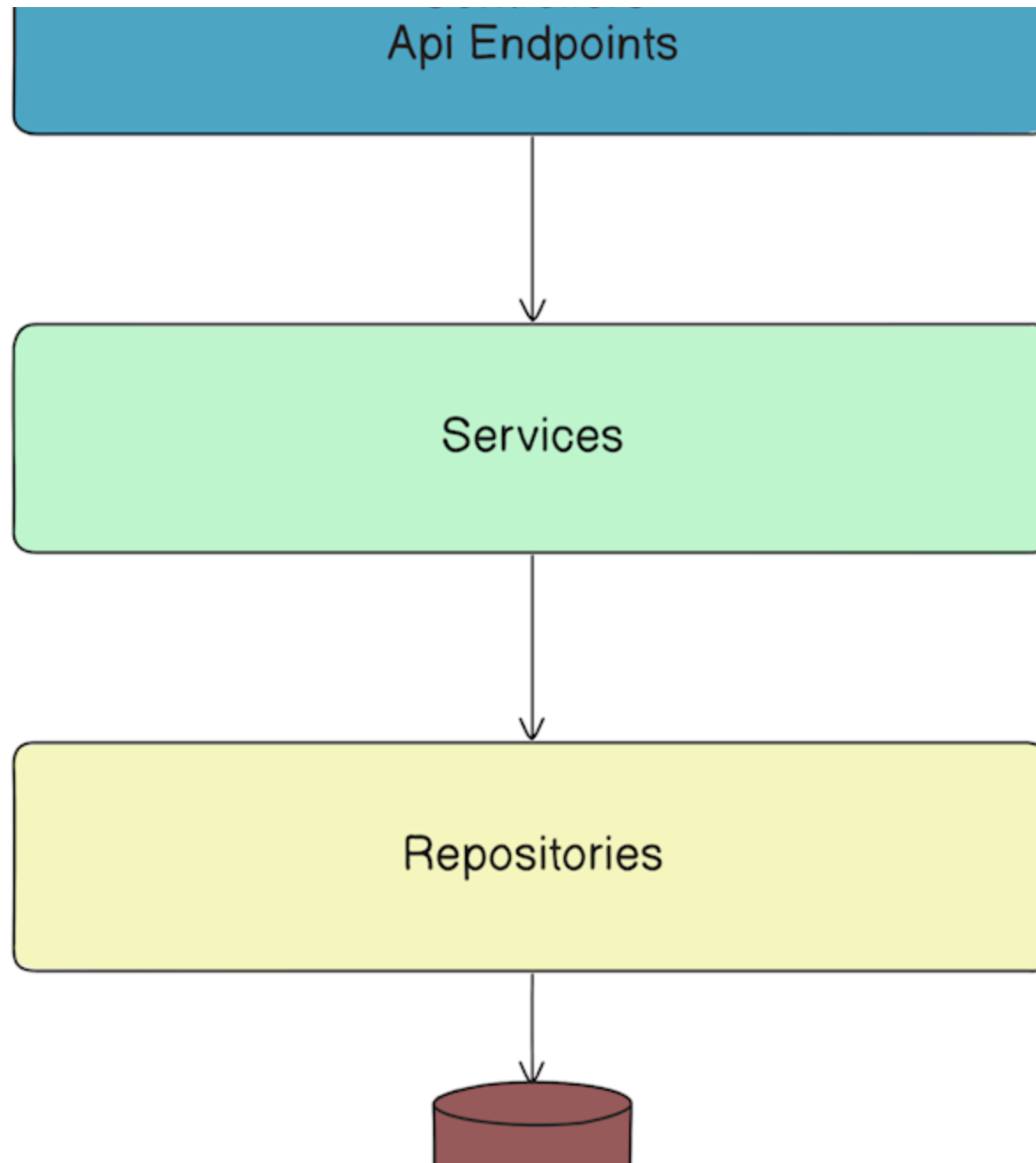


Anton Martyniuk

- /Repositories
- /Models



Anton Martyniuk



**Anton Martyniuk**

The main reason N-Layered Architecture is so popular is that it's easy to understand. If you join a new team or project, you quickly know where to put new code, making onboarding for developers easy.

It enforces a basic separation of concerns, and for small to mid-size CRUD applications, it's an easy default.

A typical .NET Web API project using N-Layered might look like this:

csharp

```
[ApiController]
[Route("api/shipments")]
public class ShipmentsController : ControllerBase
{
    private readonly IShipmentService _service;

    public ShipmentsController(IShipmentService service)
    {
        _service = service;
    }

    [HttpGet("{id}")]
    public async Task<IActionResult> Get(int id)
    {
        var shipment = await _service.GetShipmentByIdAsync(id);
```



Anton Martyniuk

csharp

```
public class ShipmentService : IShipmentService
{
    private readonly IShipmentRepository _repository;

    public ShipmentService(IShipmentRepository repository)
    {
        _repository = repository;
    }

    public async Task<ShipmentDto> GetShipmentByIdAsync(int id)
    {
        return await _repository.GetByIdAsync(id);
    }
}
```

csharp

```
public class ShipmentRepository : IShipmentRepository
{
    private readonly ShipmentDbContext _dbContext;

    public ShipmentRepository(ShipmentDbContext dbContext)
    {

```



Anton Martyniuk

```
public async Task<ShipmentDto> GetByIdAsync(int id)
{
    return await _dbContext.Shipments
        .Where(s => s.Id == id)
        .Select(s => new ShipmentDto { Number = s.Number, OrderId = s.OrderId })
        .FirstOrDefaultAsync();
}
```

While this approach may seem simple, it has potential problems that you might encounter in your codebases.

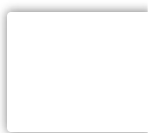
1. Fat Controllers and Fat Services

One of the most common problems with N-Layered Architecture is that Controllers and Services tend to grow rapidly as business requirements evolve.

What starts as a simple CRUD operation with 4 methods, quickly becomes a large class handling dozens of responsibilities.

Here's what a typical `ShipmentsController` looks like after a few months or years of development:

csharp





Anton Martyniuk

```
{  
    private readonly IShipmentService _shipmentService;  
  
    public ShipmentsController(IShipmentService shipmentService)  
    {  
        _shipmentService = shipmentService;  
    }  
  
    [HttpGet("{id}")]  
    public async Task<IActionResult> GetShipment(int id);  
  
    [HttpGet("user/{userId}")]  
    public async Task<IActionResult> GetShipmentsByUser(int userId);  
  
    [HttpGet("date-range")]  
    public async Task<IActionResult> GetShipmentsByDateRange(DateTime from, DateTime to);  
  
    [HttpPost]  
    public async Task<IActionResult> CreateShipment(CreateShipmentRequest request);  
  
    [HttpPut("{id}")]  
    public async Task<IActionResult> UpdateShipment(int id, UpdateShipmentRequest request);  
  
    [HttpPatch("{id}/status")]  
    public async Task<IActionResult> UpdateShipmentStatus(int id, ShipmentStatus status);  
  
    [HttpDelete("{id}")]
```



Anton Martyniuk

```
public async Task<IActionResult> TrackShipment(int id),  
  
    [HttpPost("{id}/cancel")]  
    public async Task<IActionResult> CancelShipment(int id);  
  
    [HttpPost("{id}/approve")]  
    public async Task<IActionResult> ApproveShipment(int id);  
}
```

Yes, you can separate these endpoints into multiple Controllers, but adding just another method feels easier for a developer than extracting a new Controller. Service and Repository classes can grow even worse than this.

2. Too Many Small Services and Repositories

As your domain grows, you face a critical decision: **should you create a Service and Repository for each entity?**

When dealing with `Shipments`, `ShipmentItems`, and `Orders`, following the traditional N-Layered approach leads to:

csharp



Anton Martyniuk

```
Task<IEnumerable<ShipmentDto>> GetAllAsync();  
// ...  
}  
  
public interface IShipmentItemRepository  
{  
    Task<ShipmentItemDto> GetByIdAsync(int id);  
    Task<IEnumerable<ShipmentItemDto>> GetByShipmentIdAsync(int shipmentId);  
    // ...  
}  
  
public interface IOrderRepository  
{  
    Task<OrderDto> GetByIdAsync(int id);  
    Task<IEnumerable<OrderDto>> GetByUserIdAsync(int userId);  
    // ...  
}
```

But what happens when you need to fetch related entities together? For example:

- Get a shipment with all its items
- Get an order with its associated shipments
- Get shipment historical data that loads multiple entities



Anton Martyniuk

implement a new feature, you start thinking where to add a new method in one of the three repositories.

3. Weak Business Logic and Hard to Test Solutions

Business rules are often scattered across multiple service classes, making it challenging to identify and understand them.

In N-Layered Architecture, there are no strict rules; you can bypass any layer and use the Repository in the Controller directly, for example. Additionally, you can omit the interfaces and pass implementations directly to layers, which leads to hard-to-test code.

The result?

You end up with awkward solutions:

- **Fat repositories** that know too much about other entities
- **Too many small repositories** that have 1-2 methods
- **Service methods** that orchestrate multiple repositories, leading to N+1 queries
- **Duplicate logic** across different services when they need similar cross-entity operations



Anton Martyniuk

harder to understand and modify as business requirements change.

While this pattern isn't "bad" for small or simple projects, it's rarely the best option for teams who want faster changes, cleaner code, and true modularity.

Many .NET teams are now building more complex systems, like Modular Monoliths, DDD, or microservices. In my experience, I have built a lot of projects with N-Layered Architecture and I realized that it's holding me back.

This is where I began adopting Clean Architecture in my projects a few years ago.

Clean Architecture

I know some teams use N-Layered Architecture in microservices for CRUD operations and maintain a high level of discipline in writing quality code and following established rules. But this is not the case for most teams.

Clean Architecture gives you this discipline out of the box.

Clean Architecture aims to separate the application's concerns into distinct layers, promoting high cohesion and low coupling.

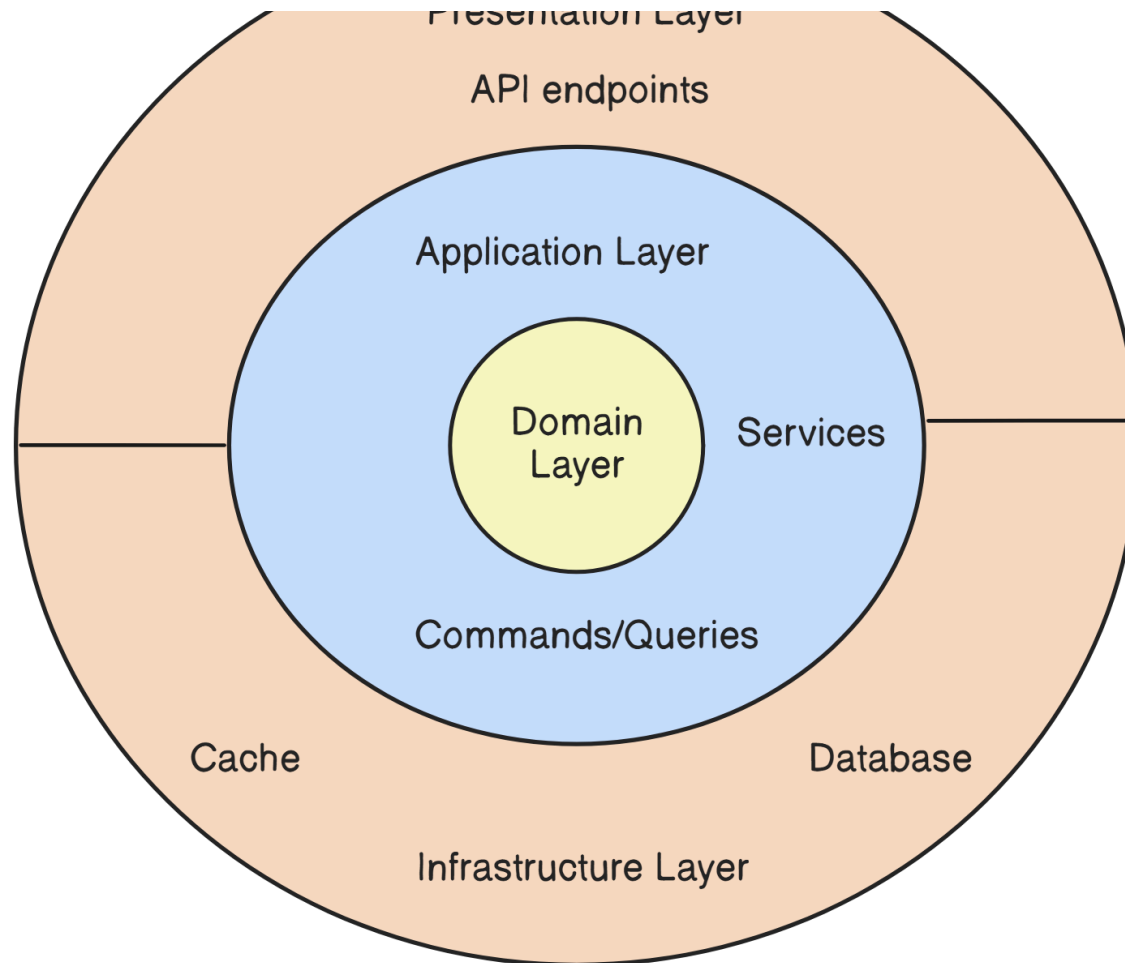


Anton Martyniuk

- **Application:** implementation of business use cases (like the Service Layer in N-Layered).
- **Infrastructure:** implementation of external dependencies like database, cache, message queue, authentication provider, etc.
- **Presentation:** implementation of an interface with the outside world, like WebApi, gRPC, GraphQL, MVC, etc.



Anton Martyniuk



It's common to see solutions organized as:

- Domain



Anton Martyniuk

- /QueryHandlers
- /Commands
- /CommandHandlers
- Infrastructure
- Presentation

Clean Architecture solves many drawbacks of N-Layered Architecture:

1. **Separation of Concerns:** Clean Architecture enforces a clear separation between different layers of the application. All layers point inside: you can't call Application use cases from the Domain Layer. This makes the codebase more maintainable and easier to understand.
2. **Testability:** By isolating the business logic from the infrastructure and UI, Clean Architecture makes it easier to write unit tests. The core of the application (the use cases and entities) can be tested without worrying about external dependencies and concrete implementations.
3. **Flexibility:** Clean Architecture allows you to change the technology stack (e.g., switching from one database provider to another) with minimal impact on the core business logic. This flexibility is achieved by abstracting infrastructure concerns behind interfaces that the core application depends on.



Anton Martyniuk

From my own experience, I can tell you that Clean Architecture is highly adaptable and can survive changing business requirements while maintaining the same code quality. While in N-Layered, it can be challenging to maintain the same level of code quality in the face of ever-evolving changes.

The typical Clean Architecture anticipates using MediatR or [manual handlers](#) in the API endpoints:

csharp

```
public class CreateShipmentEndpoint : IEndpoint
{
    public void MapEndpoint(WebApplication app)
    {
        app.MapPost("/api/shipments", Handle);
    }

    private static async Task<IResult> Handle(
        [FromBody] CreateShipmentRequest request,
        IValidator<CreateShipmentRequest> validator,
        IMediator mediator,
        CancellationToken cancellationToken)
    {
        var validationResult = await validator.ValidateAsync(request, cancellationToken);
        if (!validationResult.IsValid)
        {
            return Results.ValidationProblem(validationResult.ToDictionary());
        }
    }
}
```



Anton Martyniuk

```
var response = await mediator.Send(command, cancellationToken),
if (response.IsError)
{
    return response.Errors.ToProblem();
}

return Results.Ok(response.Value);
}
}
```

API endpoints are placed inside a Presentation project and call Application Layer Handlers (or Services).

In this case, `CreateShipmentCommandHandler` uses `IShipmentRepository` and knows nothing about the actual `ShipmentRepository` implementation in the Infrastructure layer:

csharp

```
internal sealed class CreateShipmentCommandHandler(
    IShipmentRepository repository,
    IUnitOfWork unitOfWork,
    ILogger<CreateShipmentCommandHandler> logger)
: IRequestHandler<CreateShipmentCommand, ErrorOr<ShipmentResponse>>
{
    public async Task<ErrorOr<ShipmentResponse>> Handle(
        CreateShipmentCommand request,
        CancellationToken cancellationToken)
```



Anton Martyniuk

```
        logger.LogInformation("Shipment for order '{OrderId}' is already created", request.OrderId);  
        return Error.Conflict($"Shipment for order '{request.OrderId}' is already created");  
    }  
  
    var shipmentNumber = new Faker().Commerce.Ean8();  
    var shipment = request.MapToShipment(shipmentNumber);  
  
    await repository.AddAsync(shipment, cancellationToken);  
    await unitOfWork.SaveChangesAsync(cancellationToken);  
  
    logger.LogInformation("Created shipment: {@Shipment}", shipment);  
  
    var response = shipment.MapToResponse();  
    return response;  
}  
}
```

Pragmatic Clean Architecture

Okay, now we have a clear separation of layers that point inside, but we still face the same problem with Repositories that we had in N-Layered Architecture.



Anton Martyniuk

Yes, you heard it right: using EF Core directly in the Command Handlers without creating repositories.

But doesn't it break all the benefits Clean Architecture provides? Not exactly. Let me explain.

First, EF Core's DbContext already implements the Repository and Unit of Work patterns, as stated in the official DbContext's code summary. When we create a repository over EF Core, we create an abstraction over an abstraction, leading to over-engineered solutions.

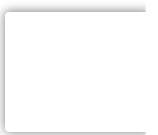
Second, how many times have you changed your database in production?

In 99% of cases, you won't need to switch the database. However, even if you switch, it's more than just changing EF Core to MongoDB, for example.

If you switch to a completely different database, it may require a complete rewrite of your existing application, as data access patterns can change significantly.

On the other hand, when you switch from one SQL database to another (for example, Postgres → SQL Server), 95% of your code in EF Core won't change.

What about testability and duplicating EF calls over different classes?





Anton Martyniuk

and on the way, as a trade-off, you can create a repository for a few queries to avoid code duplication.

There is no single correct way to write the software; you need to pick whatever works best in each particular project and case.

That's why using EF Core directly in the application use cases is a trade-off that gives more advantages than disadvantages.

Here is how the `CreateShipmentCommandHandler` changes when we get rid of repositories:

csharp

```
internal sealed class CreateShipmentCommandHandler(
    ShipmentsDbContext context,
    ILogger<CreateShipmentCommandHandler> logger)
    : IRequestHandler<CreateShipmentCommand, ErrorOr<ShipmentResponse>>
{
    public async Task<ErrorOr<ShipmentResponse>> Handle(
        CreateShipmentCommand request,
        CancellationToken cancellationToken)
    {
        var shipmentAlreadyExists = await context.Shipments.AnyAsync(x => x.OrderId == request.OrderId);
        if (shipmentAlreadyExists)
        {
            logger.LogInformation("Shipment for order '{OrderId}' is already created", request.OrderId);
            return Error.Conflict($"Shipment for order '{request.OrderId}' is already created");
        }
    }
}
```



Anton Martyniuk

```
var shipment = request.MapToShipment(shipmentNumber),  
  
await context.Shipments.AddAsync(shipment, cancellationToken);  
await context.SaveChangesAsync(cancellationToken);  
  
logger.LogInformation("Created shipment: {@Shipment}", shipment);  
  
var response = shipment.MapToResponse();  
return response;  
}  
}
```

Clean Architecture and Rich Domain Model

Okay, we now have strict rules (you can further secure them with Architecture Tests), but what about business logic, which is often scattered and duplicated across service classes, leading to hard-to-maintain solutions? This is mainly due to the use of anemic Domain Entities.

Let's explore an example of such an anemic model:

csharp

```
public class Shipment  
{
```



Anton Martyniuk

```
public Address Address { get; set; }  
public string Carrier { get; set; }  
public string ReceiverEmail { get; set; }  
public ShipmentStatus Status { get; set; }  
public List<ShipmentItem> Items { get; set; }  
public DateTime CreatedAt { get; set; }  
public DateTime? UpdatedAt { get; set; }  
}
```

When you look at this entity, you can only see properties with public getters and setters. This entity doesn't provide any information about how a Shipment's Status can change or what checks should be performed when adding Shipment Items.

As all properties are exposed with a public setter, nothing stops a developer from updating the property from another class, bypassing any checks in the Application Layer.

To address this issue, Clean Architecture has evolved as developers started to use the Rich Domain Model. A rich domain model enforces encapsulating business logic into Domain Entities.

csharp

```
public class Shipment  
{  
    private readonly List<ShipmentItem> _items = [];
```



Anton Martyniuk

```
public string OrderId { get; private set; }
public Address Address { get; private set; }
public string Carrier { get; private set; }
public string ReceiverEmail { get; private set; }
public ShipmentStatus Status { get; private set; }

public IReadOnlyList<ShipmentItem> Items => _items.AsReadOnly();

public DateTime CreatedAt { get; private set; }
public DateTime? UpdatedAt { get; private set; }

private Shipment()
{
}

public static Shipment Create(
    string number,
    string orderId,
    Address address,
    string carrier,
    string receiverEmail,
    List<ShipmentItem> items)
{
    var shipment = new Shipment { ... };
    shipment.AddItem(items);

    return shipment;
}
```




Anton Martyniuk

```
public void AddItems(List<ShipmentItem> items) { ... },
public void RemoveItem(ShipmentItem item) { ... };
public void UpdateAddress(Address newAddress) { ... };

public ErrorOr<Success> Process()
{
    if (Status is not ShipmentStatus.Created)
    {
        return Error.Validation("Can only update to Processing from Created status");
    }

    Status = ShipmentStatus.Processing;
    UpdatedAt = DateTime.UtcNow;

    return Result.Success;
}

public ErrorOr<Success> Dispatch()
{
    if (Status is not ShipmentStatus.Processing)
    {
        return Error.Validation("Can only update to Dispatched from Processing status");
    }

    Status = ShipmentStatus.Dispatched;
    UpdatedAt = DateTime.UtcNow;
}
```



Anton Martyniuk

```
public ErrorOr<Success> Transact() { ... },  
public ErrorOr<Success> Deliver() { ... };  
public ErrorOr<Success> Receive() { ... };  
public ErrorOr<Success> Cancel() { ... };  
}
```

Now, the `Shipment` class doesn't expose public setters for its properties; instead, you use a static Factory method `Create` to create a `Shipment` instance. This way, you can encapsulate any logic and validations inside this method, ensuring a single and correct way to create your entity.

Additionally, the `Shipment` class provides methods that update its state, including `AddItem`, `Process`, `Dispatch`, `Cancel`, and others. This way we encapsulate logic in a single place, eliminating business rules from scattering and duplicating across different classes.

Clean Architecture and Features Folders

The traditional approach organized code by technical concerns - separating Controllers, QueryHandlers, Services, and Repositories into different folders. However, developers quickly realized this made it difficult to understand and modify features, as related code was scattered across multiple folders.

Clean Architecture has naturally evolved from Technical Folders into Feature Folders.



Anton Martyniuk

implement the `create shipment` use case, you now have a `/features/shipments/create shipment` folder containing everything needed for that feature.

This makes the codebase more intuitive to navigate and eliminates the need to jump between a few projects and 5-7 files to implement a single feature.

While Clean Architecture has a lot of advantages, it's not a silver bullet.

Here are the drawbacks of Clean Architecture:

- **Complexity:** Clean Architecture introduces multiple layers and abstractions, which can increase the complexity of the codebase, especially for small projects. Developers may find it overwhelming if the architecture is applied unnecessarily to simple applications.
- **Overhead:** The separation of concerns and the use of interfaces can lead to additional boilerplate code, which might slow down the development process. This overhead can be particularly noticeable in smaller projects where the benefits of Clean Architecture may not be as noticeable.
- **Learning Curve:** For developers who are not familiar with Clean Architecture, there is a steep learning curve. Understanding the principles and correctly applying them can take time, especially for those new to software architecture patterns.
- **Initial Setup Time:** Setting up a Clean Architecture project from scratch requires careful planning and organization. The initial setup time can be longer compared to the N-Layered Architecture.



Anton Martyniuk

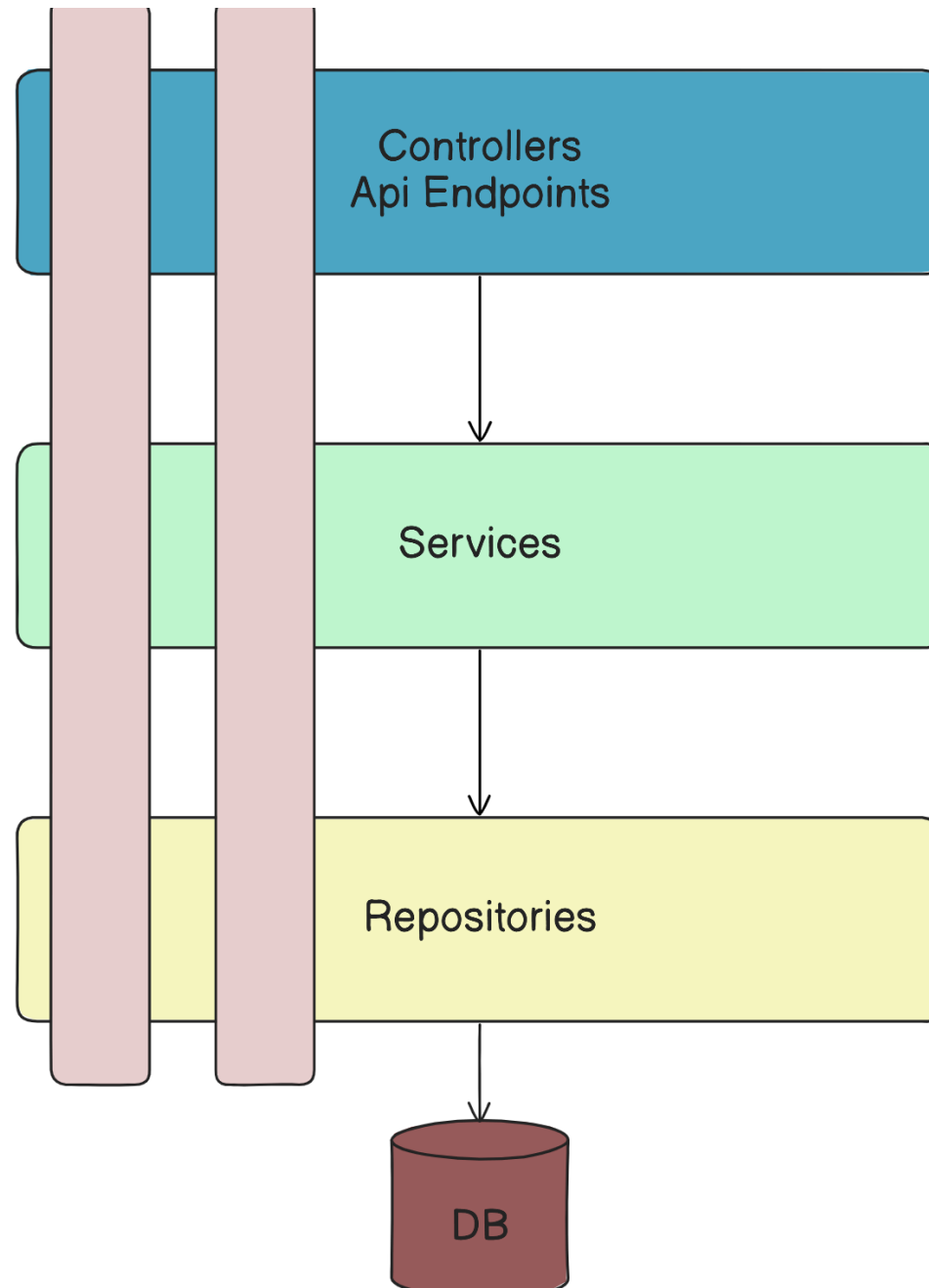
Vertical Slice Architecture

[Vertical Slice Architecture](#) is an extremely popular way to structure your projects nowadays. It strives for high cohesion within a slice (feature) and loose coupling between slices.

It structures an application by features rather than technical layers. Each slice encapsulates all aspects of a specific feature, including the API, business logic, and data access.



Anton Martyniuk





Anton Martyniuk



Anton Martyniuk

```
> Abstract
> Extensions
  C# EndpointResultsExtensions.cs
  C# MapEndpointExtensions.cs
> Shipments
  > CancelShipment
  > CreateShipment
    C# CreateShipment.cs
    C# CreateShipment.Mapping.cs
    C# CreateShipment.Validators.cs
  > DeliverShipment
    C# DeliverShipment.cs
  > DispatchShipment
  > GetShipmentByNumber
  > ProcessShipment
  > ReceiveShipment
  > Shared
  > TransitShipment
```



Anton Martyniuk

CRUD services with a few endpoints, it's more than enough to inject DbContext directly into the API endpoints.

For more complex projects, I create Handlers for each use case. Before I used MediatR commands and queries for this, however, I now tend to create my [handlers manually](#).

VSA has the following advantages:

- **Feature Focused:** Changes are isolated to specific features, reducing the risk of unintended side effects.
- **Scalability:** easier to scale development by allowing other developers and teams to work on different features independently.
- **Flexibility:** allows using different technologies or approaches within each slice as needed.
- **Maintainability:** easier to navigate in the solution, understand and maintain since all aspects of a feature are contained within a single slice.
- **Reduced Coupling:** minimizes dependencies between different slices.

Here are the drawbacks of Vertical Slice Architecture:

- **Potential Code Duplication:** potential for code duplication across slices.



Anton Martyniuk

- **Large number of classes and files:** A large application can have a lot of vertical slices, each containing multiple small classes.

With the first two disadvantages, you can deal with them by carefully designing your architecture. For example, you can extract common functionality into its own classes.

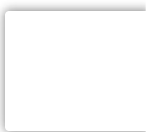
To manage cross-cutting concerns, such as error handling, logging, and validation, you can use ASP.NET Core middlewares. A well-structured folder could solve the third disadvantage.

On the one hand, Clean Architecture provides a clear separation between the different layers of the application. But on the other hand, you need to navigate across multiple projects to explore the implementation of a single use case.

One of the best aspects of Clean Architecture is that it provides a Domain-centric design for your application, significantly simplifying the development of complex domains and projects.

Vertical Slice Architecture, on the other hand, allows you to organize your code in a way that offers rapid navigation and development. A single use case implementation is one place.

What if we could take the best parts of both worlds and combine Clean Architecture with Vertical Slices?





Anton Martyniuk

transformation into Vertical Slice Architecture. Original Clean Architecture wasn't always about separating your solution into projects, but rather about your classes and their relationships.

The main point is that classes of inner layers can't call classes of outer layers. With Vertical Slice Architecture, you can achieve the same but with fewer projects.

I found that combining Clean Architecture with Vertical Slices is an excellent architecture design for complex applications. In small applications or in applications that don't have complex business logic, you can use Vertical Slices without a Clean Architecture.

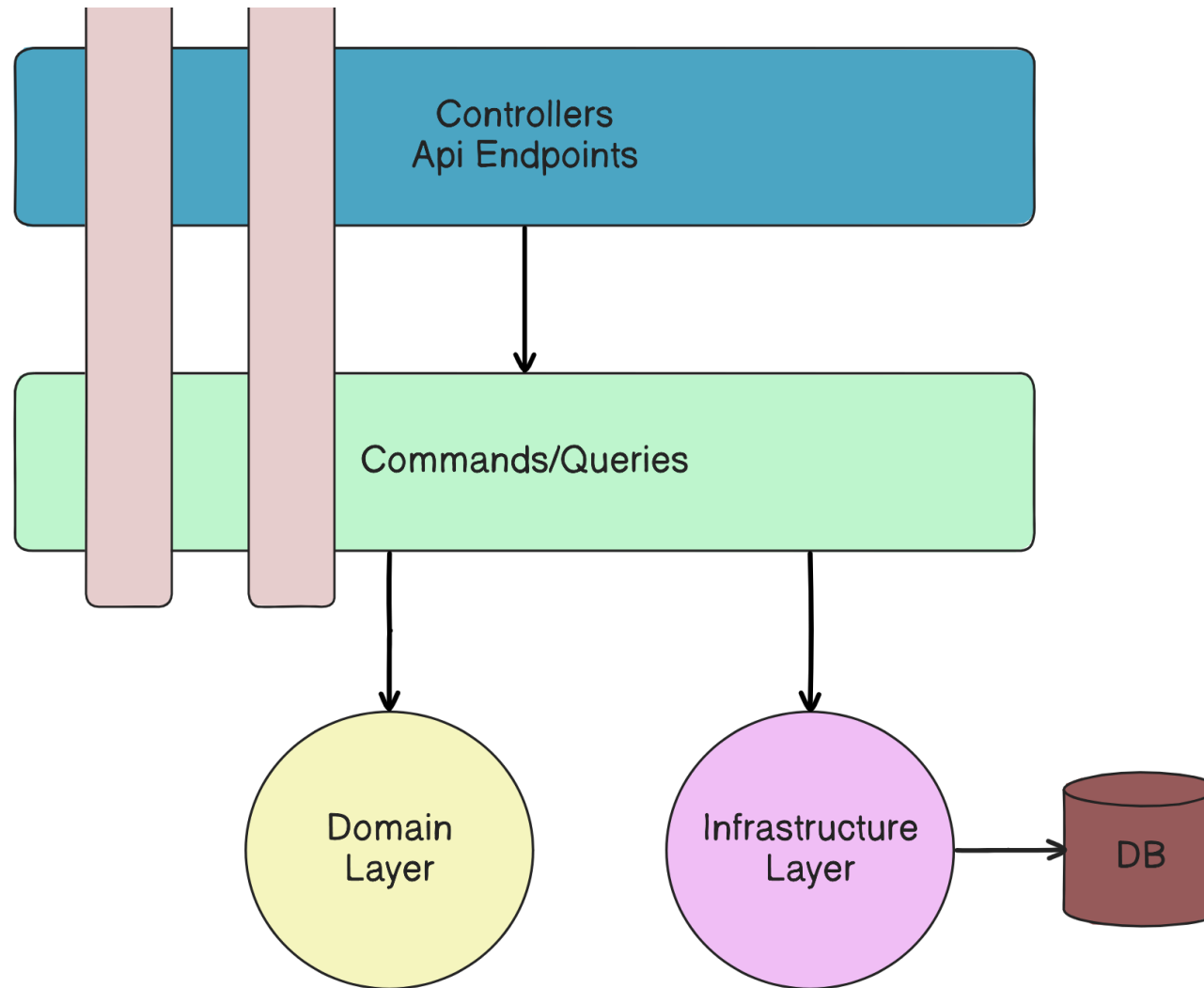
As a core, I use Clean Architecture layers and combine them with Vertical Slices.

Here is how the layers are being modified:

- **Domain:** contains core business objects such as entities (remains unchanged).
- **Infrastructure:** implementation of external dependencies like database, cache, message queue, authentication provider, etc (remains unchanged).
- **Application and Presentation** Layers are combined with Vertical Slices.



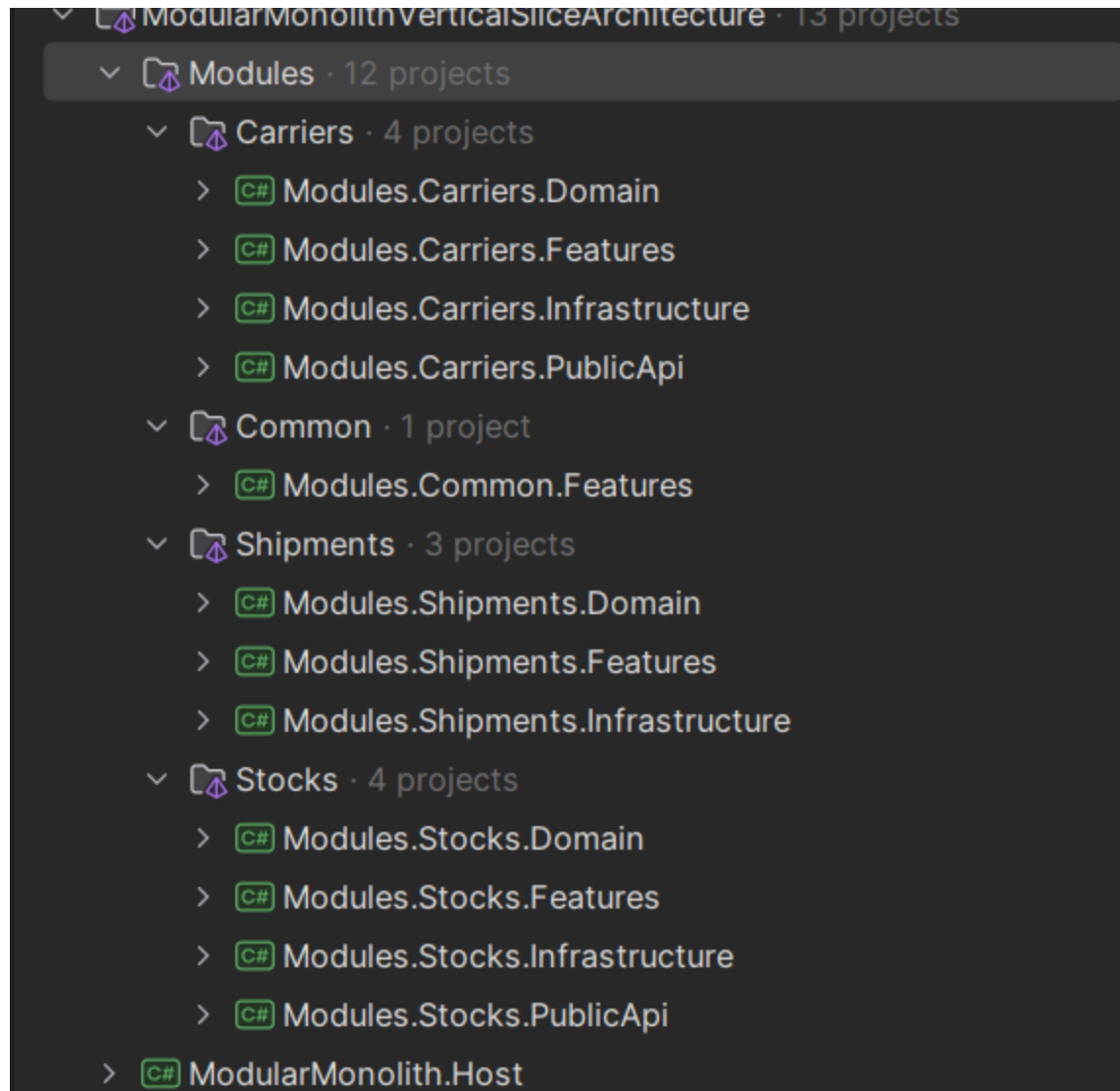
Anton Martyniuk



Here is what such a project structure looks like:



Anton Martyniuk





Anton Martyniuk

integrations, you can omit the infrastructure project.

The implementation of the `CreateShipment` use case is actually the same as in the Clean Architecture solution:

csharp

```
public class CreateShipmentEndpoint : IEndpoint
{
    public void MapEndpoint(WebApplication app)
    {
        app.MapPost("/api/shipments", Handle);
    }

    private static async Task<IResult> Handle(
        [FromBody] CreateShipmentRequest request,
        IValidator<CreateShipmentRequest> validator,
        IMediator mediator,
        CancellationToken cancellationToken)
    {
        var validationResult = await validator.ValidateAsync(request, cancellationToken);
        if (!validationResult.IsValid)
        {
            return Results.ValidationProblem(validationResult.ToDictionary());
        }

        var command = request.MapToCommand();
```

**Anton Martyniuk**

```
        ^  
        return response.Errors.ToProblem();  
    }  
  
    return Results.Ok(response.Value);  
}  
}
```

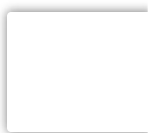
Vertical Slices, together with Clean Architecture, require some time for new developers to onboard, but significantly reduce the time it takes for them to understand the actual use cases and domain.

The Best Architecture to use in 2025

After exploring N-Layered, Clean Architecture, and Vertical Slice Architecture, you might wonder: which one should you choose for your next project? The answer depends on your team size, project complexity, and business requirements.

When to Use N-Layered Architecture

Best for:





Anton Martyniuk

- Projects with straightforward business logic
- When you need to get something working quickly and developers are already familiar with N-Layered

Choose N-Layered when:

- Your application is primarily data-driven with minimal business rules
- You have junior developers who need a familiar, easy-to-understand structure
- The project scope is well-defined and unlikely to change significantly
- You're building prototypes or proof-of-concept applications

When to Use Clean Architecture

Best for:

- Medium to large teams (4-10 developers)
- Complex business domains with rich logic
- Long-term projects that need to adapt to changing requirements



Anton Martyniuk

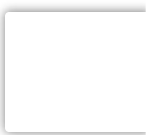
- You have complex business rules that need to be isolated and protected
- Your team has experience with architectural patterns
- You need to integrate with multiple external systems
- The project will be maintained and extended over several years
- Building a Monolith or Modular Monolith application

When to Use Vertical Slice Architecture

Best for:

- Teams of any size that value fast development
- Feature-focused development workflows
- Applications with many independent features
- Projects where different features might use different technologies

Choose Vertical Slice Architecture when:





Anton Martyniuk

- You need to onboard new developers quickly into your domain
- Different parts of your application have different complexity levels

When to Use Clean Architecture + Vertical Slices

Best for:

- Medium or Large applications
- Teams that want both structure and flexibility
- Projects with rich business domains and many features
- Applications that need to scale both in terms of code and team size

Choose this combination when:

- You have a complex domain that benefits from Clean Architecture's structure
- You want the development speed benefits of Vertical Slices
- Your team has experience with both architectural patterns
- You're building a system that will grow significantly over time



Anton Martyniuk

My Recommendation for 2025

For most new projects starting in 2025, I recommend **Clean Architecture combined with Vertical Slices**.

This approach gives you:

- **Structure when you need it:** Clean Architecture provides solid foundations for complex business logic
- **Speed when you want it:** Vertical Slices allow rapid feature development
- **Flexibility for the future:** Easy to add new and change existing features
- **Team scalability:** Different teams can work on different features independently

The key is choosing an approach that matches your team's skills, project complexity, and business needs today while keeping future growth in mind.

Start with what makes sense for your current situation:

- **Working on an existing project?** No need to change N-Layered for anything else until you encounter situations where adding new features takes too much effort and time
- **Building a Modular Monolith?** Go with Clean Architecture + Vertical Slices
- **Want maximum development speed?** Choose pure Vertical Slice Architecture



Anton Martyniuk

reading the following articles:

- [The Best Ways to Structure Your Project with Vertical Slice Architecture](#)
- [The Best Way To Structure Your .NET Projects with Clean Architecture and Vertical Slices](#)
- [Building a Modular Monolith With Vertical Slice Architecture in .NET](#)
- [Refactoring A Modular Monolith Without MediatR in .NET](#)

In these articles, you will find code examples with complete projects.

Hope you find this newsletter useful. See you next time.

Enjoyed this article? Share it with your network



LinkedIn



X



Facebook



Share



Anton Martyniuk

improve your **.NET** and Architecture Skills

Join my community of **17,000+** developers and architects.

Each week you will get 1 practical tip with best practices and real-world examples.

Learn how to craft better software with source code available for my newsletter.

[Join 17,000+ Subscribers](#)

Join 17,000+ developers already reading
No spam. Unsubscribe any time.



© 2025 Anton Martyniuk. All rights reserved.

Let's connect

